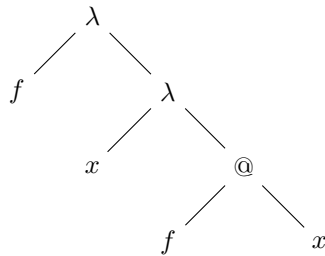


Model Solutions for Unassessed Worksheet 1

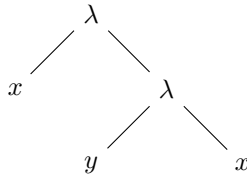
Question 1: Parsing lambda terms

Draw the syntax trees for the following λ -terms:

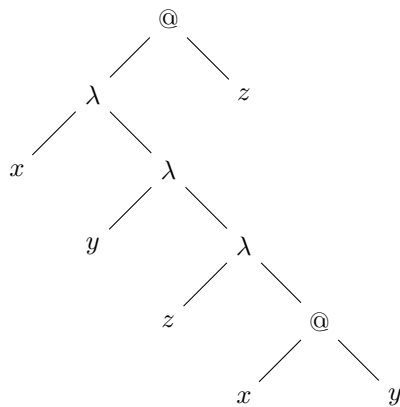
(1) $\lambda f. \lambda x. f x$



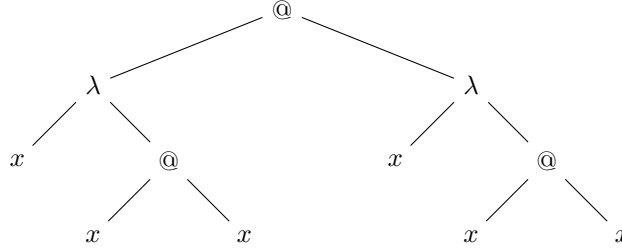
(2) $\lambda x. \lambda y. x$



(3) $(\lambda x. \lambda y. \lambda z. xy) z$



(4) $(\lambda x. xx)(\lambda x. xx)$



Question 2: Free variables and substitution

(a) Calculate the free variables of the following lambda terms.

I show the full calculation for one term, and summarize the answers for the others.

i. $\lambda x. \lambda y. x z$

$$\begin{aligned}
 FV(\lambda x. \lambda y. x z) &= FV(\lambda y. x z) - \{x\} \\
 &= FV(x z) - \{y\} - \{x\} \\
 &= FV(x) \cup FV(z) - \{y\} - \{x\} \\
 &= \{x\} \cup \{z\} - \{y\} - \{x\} \\
 &= \{z\}
 \end{aligned}$$

ii. $FV(\lambda r. f r 1) = \{f, 1\}$

iii. $FV(\lambda y. * y 2) = \{*, 2\}$

iv. $FV(\lambda y. \lambda z. g (x z) (y z)) = \{g, x\}$.

v. $FV(\lambda x. \lambda x. x) = FV(\lambda x. x) - \{x\} = \{\} - \{x\} = \{\}$

(b) Write down the result of the following capture-avoiding substitutions.

i. $(\lambda y. \lambda z. g (x z) (y z)) [g \setminus (\lambda r. f r 1)] = \lambda y. \lambda z. (\lambda r. f r 1)(x z)(y z)$

ii. $(\lambda x. \lambda y. x z) [z \setminus (\lambda y. 2)] = \lambda x. \lambda y. x(\lambda y. 2)$

iii. $(\lambda x. \lambda x. x) [x \setminus (\lambda y. * y 2)] = \lambda x. \lambda x. x$

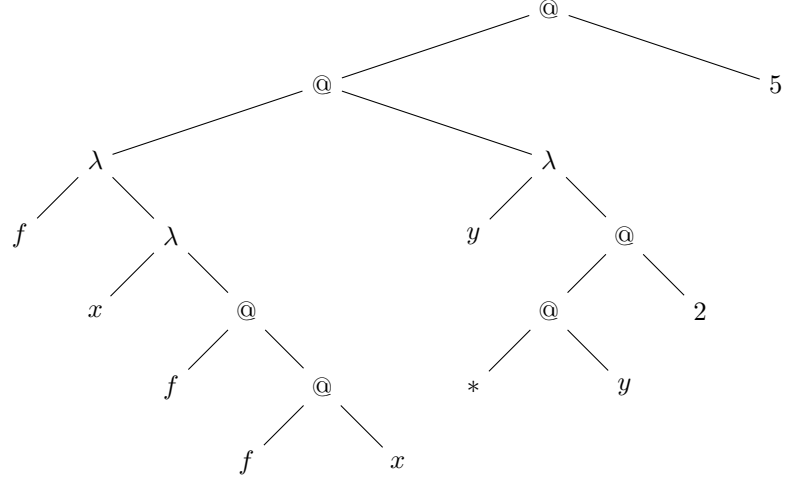
iv. $(\lambda x. \lambda y. x z) [z \setminus (y 2)] = \lambda x. \lambda w. x(y 2)$

[N.B. we have renamed the binding occurrence of y .]

Question 3: beta reduction

(a) Consider the lambda term $(\lambda f. \lambda x. f(fx)) (\lambda y. * y 2) 5$.

i. Draw its abstract syntax tree.



ii. β -reduce the term as far as possible.

$$\begin{array}{l}
 (\lambda f. \lambda x. f(fx)) (\lambda y. * y 2) 5 \\
 \xrightarrow{\beta} (\lambda x. (\lambda y. * y 2) ((\lambda y. * y 2) x)) 5 \quad \text{by applying the } \lambda f \text{ function to its argument} \\
 \xrightarrow{\beta} (\lambda y. * y 2) ((\lambda y. * y 2) 5) \quad \text{by applying the } \lambda x \text{ function to its argument} \\
 \xrightarrow{\beta} (\lambda y. * y 2) (* 5 2) \quad \text{by applying the second } \lambda y \text{ function to } 5 \\
 \xrightarrow{\beta} (* (* 5 2) 2) \quad \text{by applying the } \lambda y \text{ function to its argument}
 \end{array}$$

(b) Let S be the term $\lambda x. \lambda y. \lambda z. (xz)(yz)$ and K the term $\lambda x. \lambda y. x$. Reduce SKK to normal form.

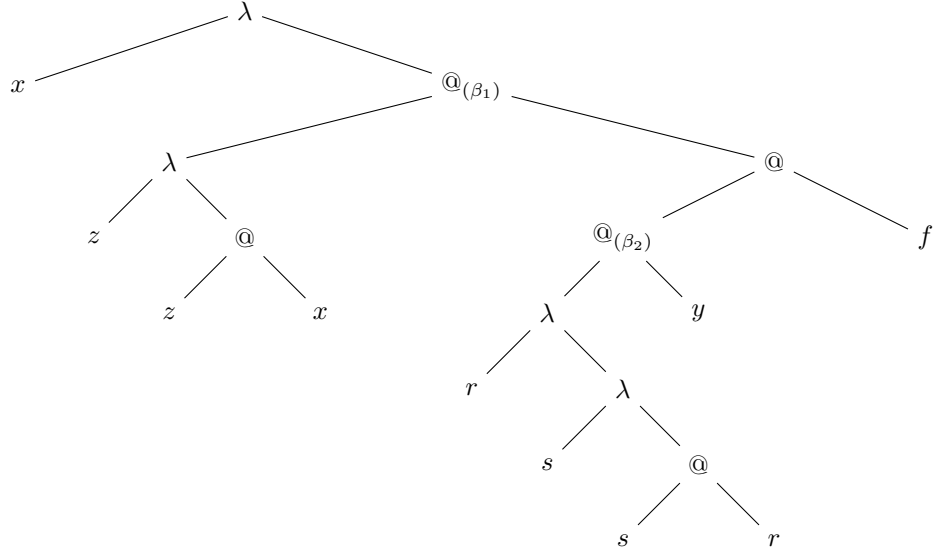
$$\begin{array}{l}
 (\lambda x. \lambda y. \lambda z. (xz)(yz)) K K \\
 \xrightarrow{\beta} (\lambda y. \lambda z. (Kz)(yz)) K \\
 \xrightarrow{\beta} \lambda z. (Kz)(Kz) = \lambda z. ((\lambda x. \lambda y. x) z) (Kz) \\
 \xrightarrow{\beta} \lambda z. (\lambda y. z) (Kz) \\
 \xrightarrow{\beta} \lambda z. z
 \end{array}$$

Question 4: beta redexes

Given below is a term M :

$$M = \lambda x. ((\lambda z. z x)((\lambda r. \lambda s. s r) y f))$$

(a) Draw the syntax tree for M .



(b) Identify two separate beta redexes in the term M . The subtrees at β_1 and β_2 in the tree above.

(c) Show the results of reducing M at each of these redexes. Call the results of these reductions N_1 and N_2 .

$$\text{Reduce } M \text{ at } \beta_1: \quad \lambda x. ((\lambda z. z x)((\lambda r. \lambda s. s r) y f)) \longrightarrow_{\beta} \lambda x. (\lambda r. \lambda s. s r) y f x \quad (\stackrel{\text{def}}{=} N_1)$$

$$\text{Reduce } M \text{ at } \beta_2: \quad \lambda x. ((\lambda z. z x)((\lambda r. \lambda s. s r) y f)) \longrightarrow_{\beta} \lambda x. (\lambda z. z x)((\lambda s. s y) f) \quad (\stackrel{\text{def}}{=} N_2)$$

(For clarity you might prefer to write N_1 as $\lambda x. ((\lambda r. \lambda s. s r) y f) x$, but the syntax tree is the same.)

In this solution set, we always underline the redex that is being reduced at each stage. This is a good idea for you to do well, in order to be clear about the reductions you are performing.

(d) Reduce N_1 and N_2 to normal forms and check if they reduce to the same normal form.

For N_1 :

$$\lambda x. (\lambda r. \lambda s. s r) y f x$$

$$\longrightarrow_{\beta} \lambda x. (\lambda s. s y) f x$$

$$\longrightarrow_{\beta} \lambda x. f y x$$

For N_2 :

$$\lambda x. (\lambda z. z x)((\lambda s. s y) f)$$

$$\longrightarrow_{\beta} \lambda x. (\lambda z. z x)(f y)$$

$$\longrightarrow_{\beta} \lambda x. f y x$$

(e) Specify if your redexes in step b are outermost or innermost redexes.

β_1 is an outermost redex because there is no other redex outside it. It is not an innermost redex.

β_2 is an innermost redex because there is no other redex inside it. It is not an outermost redex.

- (f) What are the pro's and con's of always choosing outermost redexes for beta reduction? Similarly, for innermost redexes?

Pro outermost: Reducing outermost redexes can save work (sometimes an infinite amount of work) because it avoids reducing the arguments of λ functions. If a λ function is constant in its argument, then there is no need to reduce the argument because it will get discarded.

Con outermost: If a λ function has multiple occurrences of its argument, then the argument will need to be reduced multiple times. This can add work.

Pro innermost: Every argument term is evaluated exactly once. This saves work if functions use their arguments multiple times, e.g., $\lambda y. f y y$.

Con innermost: This same feature does unnecessary work if the function doesn't use its argument at all.

- (g) Show that the term M above is β -equivalent to the following term:

$$N = \lambda x. ((\lambda z. \lambda u. \lambda v. u v z) x f y)$$

The term N can be reduced as follows:

$$\frac{N = \lambda x. ((\lambda z. \lambda u. \lambda v. u v z) x f y)}{\longrightarrow_{\beta} \lambda x. f y x} \quad \text{by applying the } \lambda z, \lambda u \text{ and } \lambda v \text{ abstraction terms}$$

Since M and N are β -equivalent to the same normal form, they are β -equivalent themselves.

Question 5: Programming with untyped λ -calculus* (hints)

- (a) Find a λ -term `iszero` such that

$$\text{iszero } \underline{n+1} =_{\beta} F$$

for all $n \in \mathbb{N}$ and $\text{iszero } \underline{0} =_{\beta} T$.

Hint: observe that $(\lambda x.F)t =_{\beta} F$ for any t , and think about the definition of $\underline{n}$.

- (b) Defining

$$\text{pred} := \lambda n. n (\lambda p. \lambda b. b (p F) (\text{succ } (p F))) (\lambda b. b \underline{0} \underline{0}) T$$

prove that `pred` satisfies

$$\text{pred } \underline{n+1} =_{\beta} \underline{n}$$

for all $n \in \mathbb{N}$, as well as $\text{pred } \underline{0} =_{\beta} \underline{0}$.

Hint: This can be done directly by induction, but it is also possible to see intuitively why it works.

- (i) If we define 'pairs' by

$$\langle s, t \rangle := \lambda b. b s t$$

then we have projection functions:

$$\pi_1 := \lambda p. p T$$

$$\pi_2 := \lambda p. p F$$

satisfying $\pi_i(\langle s_1, s_2 \rangle) =_{\beta} s_i$.

(ii) If we start from $\langle 0, 0 \rangle$ and then iterate the operation “ $\langle s, t \rangle \mapsto \langle t, \text{succ } t \rangle$ ” a total of $n + 1$ times, we end up with $\langle \underline{n}, \underline{n + 1} \rangle$.

(iii) `pred` essentially just does the above operation and then applies π_1 .

(N.B. writing this formally is still an induction on $n \in \mathbb{N}$).

(c) Hence find a λ -term `triangle` such that

$$\text{triangle } \underline{n} =_{\beta} \underline{\frac{1}{2}n(n+1)}$$

for all $n \in \mathbb{N}$. (Recall that $\frac{1}{2}n(n+1) = 0 + 1 + \dots + n$).

Hint: We might program this in Haskell with

```
triangle 0 = 0
triangle (n+1) = triangle n + n
```

or

```
triangle n = if n == 0
              then 0
              else triangle (n-1) + n
```

Hence `triangle` can be taken to be a fixed point of

$$\lambda f. \lambda x. \text{iszero } x \ \underline{0} \ (\text{add } (f(\text{pred } x)) \ x)$$